

PLLM: Private Multi-Party LLM Inference and Evidence-Bound Research Composition

Blair Hudson
deployscience labs
blair@deployscience.com

23 September 2026

Abstract

PLLM is a private LLM multi-party inference runtime and extensible autonomous research harness. Its public-weight path uses trusted Client, non-colluding Preparation, and Inference: offline one-use $Wr - s$ corrections permit online masked linear work while Client retains plaintext, state, and decoding. Model-neutral compilation binds typed component composition to a plan; bounded search and matched evidence enable comparison of eligible candidates. We map six cited research directions to independently implemented *portions* of PLLM, not complete reproductions. Historical CPU-loopback measurements cover nine Qwen2.5-0.5B baseline responses; optional verified linear work has only a separate one-run tiny-model matched control. Garbling and cache components lack whole-model comparative measurements. Thus these data neither establish economic value nor rank PLLM against external state of the art. We define comparability and metric gates required to make such a claim.

Introduction

Ordinary hosted inference gives one operator the user’s plaintext prompt, token identities, intermediate state, and generated output. Transport encryption protects these values in transit but not from the endpoint performing inference. Private neural-inference systems distribute computation using homomorphic encryption, secret sharing, or secure two-party computation (Huang et al. 2022; Chen et al. 2022; Hao et al. 2022).

PLLM combines a prepared masked-linear runtime with model-neutral compilation and an extensible autonomous research harness. Its automation generates bounded valid candidates and repeats experiments; researchers still implement methods independently. We contribute (1) one-use corrections and explicit trust boundaries, (2) canonical component-bound schedules with fail-closed coverage, and (3) evidence rules and metrics for comparing eligible alternatives. This systems report is **not** a security proof or a matched external SOTA benchmark.

Prepared inference protocol

Roles and application boundary

The **Client** is trusted. It owns plaintext input and output, tokenization, one-time root seeds and masks, private activation scales, attention and other nonlinear state, sampling, and decoding. For public runtime bundles it also evaluates token lookup and the output head locally. The optional `pllm gateway` runs on loopback inside this boundary. Applications submit ordinary Responses

API or Chat Completions API requests to that gateway, which invokes the private PLLM client path. Preparation and Inference are protocol services, not application-facing OpenAI-compatible endpoints.

Preparation holds the public body and offline seeds; it must erase masks and not collude with **Inference**, which holds the same body and performs online matrix work. Both roles must follow the protocol.

Offline preparation and online use

For public quantized matrix W and private integer activation x , the Client sends Preparation a fresh root seed and row count for each remote stage. Domain-separated expansion binds model, stage, quantization, arithmetic, and inventory identity to fresh input mask r , output mask s , and ticket per row. Preparation computes

$$c = Wr - s$$

and pushes c to Inference. The latter acknowledges stage batches and seals the inventory after every stage loads. It sees no root seed; Client sees no c .

Online, the Client sends Inference a one-use ticket and masked activation $x - r$. Inference atomically consumes the matching correction and returns

$$W(x - r) + c = W(x - r) + (Wr - s) = Wx - s.$$

The Client adds s and center-decodes Wx . Preparation receives no online message. Fresh pseudorandom r masks x from Inference, while fresh s masks the correction.

Reusing a row would reveal relations between activations, so reuse is forbidden.

Inventory lifecycle

Inventory commits to model/body, stage set, quantization, ring, and attempt budget. Responses atomically reserve disjoint rows; cancellation, failure, replay, and early completion burn unused reservations. Restart, replacement, and idle expiry discard in-memory inventory.

Prefill packs stage tickets and masked rows; decode reuses a persistent authenticated connection with fresh tickets. Each stage selects an exact 2^{16} , 2^{24} , or 2^{32} ring from its signed output bound.

Threat model and limits

Inference does not receive literal prompt text, token IDs, decoded output, root seeds, masks, private activation scales, local attention state, or sampling choices. Preparation receives seeds and can derive masks, but does not receive online masked activations. Under fresh pseudorandom masks, authenticated channels, protocol-conforming execution, Preparation erasure, and non-collusion, neither remote role alone has both views needed to reconstruct client activations.

The split fails if Preparation and Inference collude: Preparation’s r and Inference’s $x - r$ reconstruct x . Two services controlled by one operator do not establish meaningful non-collusion. Self-hosting Preparation keeps its trust in the client environment, but co-locating all roles is only a functional topology, not evidence of independent operators.

Both services see public model identity, tensor shapes, timing, traffic volume, and approximate sequence length. Authentication does not prove correct execution, independence, secure deletion, or side-channel resistance. Arbitrary malicious participants are out of scope; erasure of Preparation’s Python memory is not established. The local gateway and host remain trusted.

Implementation status

Prepared runtime

The `pllm.run` Python distribution includes native Rust kernels. Python owns checkpoint import, protocol, gateway, and state; Rust executes immutable, validated integer matrices, bounded codecs, and CPU dispatch. Public online execution uses prepared masks, **not** homomorphic encryption. Attention, normalization, token boundaries, nonlinear state, and sampling remain with Client. Historical prepared generation covers Qwen2.5-0.5B; it does not make the client-local regions protected components.

Semantic IR and compiler

Adapters lower Qwen2, dense Qwen3, and bounded Qwen3.5, Phi, and Gemma text configurations into a

model-neutral IR of operators, layers, and persistent state. Compiler passes avoid adapter-specific node names. Semantic lowering alone proves neither checkpoint import nor executable or private coverage.

For untransformed plans, baseline components schedule independent weighted operators sharing an input and order local operators by dependency. Binding checks tokenizer, runtime, local tensors, quantized stages, scales, and preparation commitments. A plan-bound session enforces feedback and bounds. The pinned Qwen2.5-0.5B checkpoint passes a **clear native-kernel** prefill-to-decode check; historical masked-protocol measurements are separate. Tiny dense Qwen3 uses the same compiled binding. Verified compiled execution, transformed cache, and protected whole-decoder composition remain incomplete.

Cited work versus runnable baseline

Table 1 maps **independent PLLM adaptations**, not original authors’ code, to the strongest evidence available. The pinned Qwen2.5-0.5B baseline is the only row with real-checkpoint masked-runtime timing. A paper citation is neither evidence of a whole-paper reimplement nor a cross-method speedup.

Component extensions, search, and evidence

Research methods enter capability families instead of paper-specific runtime branches. Canonical **Pipeline** identity uses serialized model and component values; the compiler binds operator, numeric, state, method, and schedule contracts to the plan and fails closed on gaps. Provider manifests are discoverable without importing code, but native provider loading is not exposed. Papers are attribution and specification, not executable dependencies.

Experiment binds composition, workload, and deployment. Grid and seeded random search generate validated candidates. Evidence records actual results; their identity includes plan and environment digests. Historical baseline measurements predate those plan locks. The experiment workflow documents the runnable SDK and CLI paths.

Optional verified linear work

FreivaldsVerify adapts Slalom’s delegated-linear verification (Tramèr and Boneh 2019) to a **different**, TEE-free trusted-client/preparation boundary. For secret uniform $a \in \mathbb{F}_{2^{32}-5}$, Client checks $\langle a, y \rangle = \langle W^T a, x \rangle$ before dequantization. Signed bounds prevent field wrap; independent checks and process-wide attempt accounting bound false acceptance. Preparation sends authenticated one-use projections bound to inventory and stage; Inference sees no challenge. Malicious Preparation and collusion remain out of scope. Verification does **not** reduce online traffic. Compiled verified execution fails closed until bound to a verifier-enforcing executor.

Table 1: Scope of independent research adaptations. The masked baseline is PLLM’s protocol, not a Slalom reproduction. Payload bytes are not response traffic or latency; no cited method has pinned-Qwen comparative evidence.

Source	PLLM part	Current measurement	Pinned Qwen?
PLLM baseline	Prepared masked linear	Nine warm responses	Yes, older revision
DASH (Sander et al. 2024)	Q7 garbled SiLU reference	Error at most 0.02285	No
ReDASH (Maurer et al. 2025)	Bounded Q7 rescale/arithmetic	Numeric tests only	No
CRT garbling (Ball et al. 2017)	Q7 multiply gadget	245,397 B scalar payload	No
Half-gates (Zahur et al. 2015)	Boolean circuit reference	Correctness tests only	No
Slalom (Tramèr and Boneh 2019)	Freivalds stage check	One-run tiny paired control	No
MPCache (Zeng et al. 2025)	KV-eviction plan pass	Plan lineage only	No

Evaluation and evidence boundaries

Method

Historical prepared runtime. docs/evidence/current-runtime-2026-09-11.json records source revision 277d19f, CPython 3.13.15, and Qwen2.5-0.5B-Instruct checkpoint revision 7ae5576 (full lock in the artifact) (Qwen Team 2024) on one Apple M5 CPU host (32 GiB). Client, Preparation, and Inference ran as separate loopback processes. One excluded warmup preceded three warm runs at each exact input length. Output was capped at 16 tokens; the 30-token cohort produced only nine. Only completed responses with settled telemetry were included. These records **predate** the semantic compiler.

Table 2: Historical cohort medians (three runs each); decimal MB is total Client traffic. TTFT starts after inventory readiness; full time includes offline preparation. On-line medians were 2.618, 3.194, and 7.442 s; offline correction pushes were 59.80, 72.88, and 252.18 MB, respectively.

In/out	TTFT, s	Full, s	Client MB
30 / 9	0.978	4.674	63.33
63 / 16	1.374	5.347	126.29
255 / 16	5.158	13.059	432.71

All nine records report zero Preparation requests and protocol operations during the online interval, and zero literal plaintext prompt or token-ID bytes in the instrumented counters. These observations validate the measured implementation path, not a general privacy theorem. Preparation remains real computation and traffic despite occurring outside online timing.

Verified tiny control. Separate docs/evidence/ records named `slalom-freivalds-tiny-baseline.json` and `slalom-freivalds-tiny-verified.json` share generated model fingerprint, workload, environment, and one completed local response per composition. The baseline took 0.666 s and verified path 0.627 s full latency. With **one sample, no warmup, synthetic weights, and no**

real-Qwen checkpoint, this comparison shows the optional verification path works in a matched tiny topology; it cannot estimate performance or prove security. Neither record contains a PlanLock digest. The functional verifier tests cover forged stage output, authenticated material import, one-use consumption, and session-wide attempt accounting; they do not establish malicious Preparation resistance.

Limits. No measured concurrent load, physical network, accelerator, model-quality study, secure-erasure study, independent operator deployment, or cross-system matched cohort exists. Offline preparation incurs substantial traffic and full latency. These results cannot be extrapolated to other models, current compiled execution, operating cost, or provider-level security.

Defining an eligible state-of-the-art comparison

PLLM’s metric definitions cover latency, throughput, communication, memory, energy, accuracy, perplexity, and cost with explicit units and optimization directions. A defined metric is **not a measured value**. The retained Qwen study reports latency and protocol traffic, but no price, energy, quality, or external-system result. Search validates candidate configurations; benchmark records preserve exact cohorts, and Pareto comparison uses declared metric directions without implicitly ranking incomparable evidence.

We define an eligible comparison as the *same* checkpoint/body, numeric policy, input and output counts, output cap, warm state, hardware/network environment, quality threshold, and compatible privacy and assurance contract. Candidates must first pass correctness and trust gates. Within that cohort, a candidate dominates another only on **measured** metrics with explicit directions and at least one strict gain. Absent measurements cannot be imputed; a cheaper response must count client work, offline rows used or burned, online compute, network, and deployment resources. This defines how a future external SOTA claim could be tested, not a claim that one currently exists.

Bounded research components

Inspired by arithmetic garbling (Sander et al. 2024; Maurer et al. 2025), a one-use Q7 SiLU region computes $q(x) = x/2 + x^2/4$ on $[-1, 1]$ with ties-to-even rounding. Exhaustive encoded-domain testing bounds absolute SiLU error by 0.02285. Evaluation is limited to 128 elements and burns digest-bound material once.

A separate region jointly garbles SiLU and Q7 multiplication without decoding intermediate labels. Selectable methods retain a dense binary-table baseline and add compact mixed-modulus multiplication based on CRT projection gadgets (Ball et al. 2017). Half-gates supply a separate experimental Boolean reference (Zahur et al. 2015). One-lane evaluator payloads measure 2,387,674 and 245,397 bytes, respectively; four independent compact lanes measure 980,573 bytes. These primitives have no whole-decoder tensor-scale execution, cryptographic review, or production-security claim and do not replace the prepared additive-masking protocol.

Conclusion

PLLM combines prepared private inference with independently implemented, composition-bound research components. Its Qwen baseline, tiny verified control, and isolated numeric/garbling tests are **different evidence levels**. Protected whole-decoder execution, matched method measurements on the pinned model, and external SOTA and economic comparisons remain open.

References

- Ball, Marshall, Tal Malkin, and Mike Rosulek. 2017. “Garbling Gadgets for Boolean and Arithmetic Circuits.” *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 565–77. <https://doi.org/10.1145/3133956.3133964>.
- Chen, Tianyu, Hangbo Bao, Shaohan Huang, et al. 2022. “THE-X: Privacy-Preserving Transformer Inference with Homomorphic Encryption.” *Findings of the Association for Computational Linguistics: ACL 2022*, 3510–20. <https://doi.org/10.18653/v1/2022.findings-acl.277>.
- Hao, Meng, Hongwei Li, Hanxiao Chen, Pengzhi Xing, Guowen Xu, and Tianwei Zhang. 2022. “Iron: Private Inference on Transformers.” *Advances in Neural Information Processing Systems 35*, 15718–31. <https://doi.org/10.52202/068431-1143>.
- Huang, Zhicong, Wen-jie Lu, Cheng Hong, and Jiansheng Ding. 2022. “Cheetah: Lean and Fast Secure Two-Party Deep Neural Network Inference.” *31st USENIX Security Symposium*, 809–26. <https://www.usenix.org/conference/usenixsecurity22/presentation/huang-zhicong>.
- Maurer, Felix, Jonas Sander, and Thomas Eisenbarth. 2025. *ReDASH: Fast and Efficient Scaling in Arithmetic Garbled Circuits for Secure Outsourced Inference*. <https://arxiv.org/abs/2506.14489>.
- Qwen Team. 2024. *Qwen2.5 Technical Report*. arXiv:2412.15115. <https://doi.org/10.48550/arXiv.2412.15115>.
- Sander, Jonas, Sebastian Berndt, Ida Bruhns, and Thomas Eisenbarth. 2024. *Dash: Accelerating Distributed Private Convolutional Neural Network Inference with Arithmetic Garbled Circuits*. <https://arxiv.org/abs/2302.06361>.
- Tramèr, Florian, and Dan Boneh. 2019. *Slalom: Fast, Verifiable and Private Execution of Neural Networks in Trusted Hardware*. <https://arxiv.org/abs/1806.03287>.
- Zahur, Samee, Mike Rosulek, and David Evans. 2015. *Two Halves Make a Whole: Reducing Data Transfer in Garbled Circuits Using Half Gates*. <https://eprint.iacr.org/2014/756>.
- Zeng, Wenxuan, Ye Dong, Jinjin Zhou, et al. 2025. *MP-Cache: MPC-Friendly KV Cache Eviction for Efficient Private Large Language Model Inference*. <https://arxiv.org/abs/2501.06807>.